

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

BACHELOR THESIS

Kinan Al-Falakh

**Automating Benchmark Analysis
Through Machine Learning**

Department of Distributed and Dependable Systems

Supervisor of the bachelor thesis: Mgr. Vojtěch Horký, Ph.D.

Study programme: Computer Science

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date

Author's signature

Dedication.

Title: Automating Benchmark Analysis Through Machine Learning

Author: Kinan Al-Falakh

Department: Department of Distributed and Dependable Systems

Supervisor: Mgr. Vojtěch Horký, Ph.D., Department of Distributed and Dependable Systems

Abstract: Regular software performance benchmarking is used to prevent silent performance deterioration, yet reliably evaluating the results remains an open challenge. This thesis employs contrastive time series representation learning via TS2Vec to automate the detection of performance anomalies in GraalVM compiler benchmarks. Raw iteration times are normalized using a log-transform followed by robust IQR scaling to handle multiplicative noise from garbage collection and compiler de-optimizations. A single dilated convolutional encoder, trained configuration-independently with hierarchical temporal contrastive loss, maps variable-length benchmark runs to fixed-size embedding vectors. Performance anomalies are then detected as unusual transitions in embedding space using cosine distance between consecutive versions. The method is evaluated on GraalVM benchmark data across multiple domain-specific configurations.

Keywords: machine learning, software benchmarking, automated analysis, contrastive learning, time series

Contents

Introduction	7
1 Background, Dataset, and Problem Statement	8
1.1 Software Performance Benchmarking	8
1.2 JVM Execution Characteristics	8
1.2.1 JIT Compilation	9
1.2.2 Garbage Collection	9
1.2.3 Typical Benchmark Execution Profile	10
1.3 The GraalVM Compiler Benchmark Results Dataset	11
1.3.1 Dataset Structure	11
1.3.2 Measurement Metadata	11
1.4 Configuration as the Unit of Analysis	12
1.4.1 Defining a Configuration	12
1.4.2 Metadata Resolution and Filtering	12
1.4.3 From Configuration to Time Series	13
1.5 Summary and Problem Formulation	13
2 Preprocessing	14
2.1 Steady-State Detection	14
2.1.1 CAS Analysis (Compilation Activity Signal)	14
2.1.2 Dynamic Parameters	15
2.2 Normalization	16
2.2.1 Noise Characteristics in the Dataset	16
2.2.2 Why Standard Normalization Fails	16
2.2.3 Log-Transform and Robust IQR Scaling	16
2.3 Per-Configuration Preprocessing Pipeline	16
3 Related Work	17
3.1 Performance Regression Detection	17
3.1.1 Statistical Approaches	17
3.1.2 Machine Learning Approaches	17
3.2 Alternative Approaches	17
3.2.1 Statistical Feature Extraction	17
3.2.2 LSTM Autoencoders	17
3.3 TS2Vec and Contrastive Learning	17
3.4 Gap: Time Series Representations for Benchmark Analysis	17
4 TS2Vec and Contrastive Learning	18
5 Anomaly Detection Pipeline and Reporting	19
Conclusion	20
Bibliography	21
List of Figures	22

List of Tables	23
List of Abbreviations	24
A Attachments	25
A.1 First Attachment	25

Introduction

Systematic testing of software performance during development is a persistent challenge. Performance anomalies, changes that silently alter performance characteristics, can go unnoticed if benchmark results are not evaluated regularly. In large-scale software such as the GraalVM compiler, where benchmark data spans years of development and tens of thousands of measurement runs, reliably automated evaluation of the results becomes essential. While collecting sufficient measurement data is itself costly, another hard problem is interpreting it: software rarely behaves in a fully deterministic manner, and non-deterministic behavior in the runtime environment introduces noise into iteration times, making it difficult to determine whether a change in measured performance reflects a genuine anomaly or is simply a consequence of the execution environment.

Existing approaches to automated performance evaluation range from statistical change detection to classical unsupervised learning. These methods typically require significant manual configuration per benchmark or assume stationary data distributions, limiting their applicability to large-scale, long-running projects.

This thesis explores whether machine learning, specifically self-supervised contrastive representation learning, can better distinguish genuine performance anomalies from noise. We build and evaluate the system on the GraalVM Compiler Benchmark Results Dataset, a data artifact collected by the D3S department at Charles University together with Oracle Labs to support research into software performance testing. We train a neural encoder to learn what benchmark execution "looks like" from unlabeled data, and detect anomalies as unusual shifts in the learned representation space. The goal is to provide performance engineers with a compact set of flagged transitions that merit further investigation, reducing the volume of results requiring manual inspection.

The remainder of this thesis is organized as follows. Chapter 1 provides background on software performance benchmarking and JVM execution characteristics, introduces the GraalVM benchmark dataset, defines the notion of a configuration, and formulates the problem addressed by this thesis. Chapter 2 describes the preprocessing pipeline, including steady-state detection and the normalization strategy used to handle multiplicative noise. Chapter 3 surveys related work in performance regression detection and time series representation learning. Chapter 4 presents the TS2Vec contrastive learning framework and how it is applied to benchmark time series. Chapter 5 describes the anomaly detection pipeline, the multi-layer filtering system, and the reporting output designed for performance engineers.

1 Background, Dataset, and Problem Statement

This chapter provides the necessary background for the remainder of the thesis. We begin with an overview of software performance benchmarking and its role in software development. We then describe the execution characteristics of the Java Virtual Machine that make benchmark interpretation challenging. Finally, we introduce the GraalVM Compiler Benchmark Results Dataset, define the notion of a *configuration* as the fundamental unit of analysis, and formulate the problem addressed by this thesis.

1.1 Software Performance Benchmarking

Software performance benchmarking is the practice of executing standardized workloads under controlled conditions to measure and compare the performance of a software system across different versions, configurations, or environments. In the context of software development, benchmarking serves a critical role: it provides an objective basis for detecting performance regressions before they reach production.

A typical benchmarking workflow consists of three stages. First, a set of representative workloads is selected and executed on the software under test. Second, performance metrics such as execution time, throughput, or resource utilization are recorded for each execution. Third, the recorded metrics are compared against a baseline, typically the measurements from a previous software version, to determine whether a statistically significant change has occurred.

Each of these stages presents its own challenges. Designing efficient measurement procedures, selecting representative workloads, and managing the cost of collecting data across many software versions all require significant effort (1). However, the evaluation and interpretation of the results poses a particular difficulty because software rarely behaves in a fully deterministic manner. Even when the same code runs on the same hardware, consecutive executions can produce measurably different results due to variability in the runtime environment, hardware state, and operating system behavior. Distinguishing a genuine performance change from such variability is one of the key challenges addressed in this thesis.

The cost of benchmarking at scale is also substantial. In large projects where benchmarks must be executed across many software versions and hardware configurations, the total measurement time can reach years of machine time. This makes it impractical to simply increase the number of repetitions until statistical significance is achieved for every transition, and further emphasizes the need for methods that can reliably evaluate benchmark results automatically.

1.2 JVM Execution Characteristics

The benchmarks used in this work execute on the Java Virtual Machine (JVM). The JVM introduces several sources of non-deterministic behavior that

directly affect benchmark measurements. Understanding these sources is essential for interpreting the data and motivating the preprocessing steps described in Chapter 2.

1.2.1 JIT Compilation

The JVM supports multiple compilation strategies. In its default mode, it uses just-in-time (JIT) compilation, where bytecode is initially interpreted and only compiled to native machine code after it has been identified as a *hot method*, one that is executed frequently enough to justify the cost of compilation. More recent versions of the GraalVM Compiler also support ahead-of-time (AOT) compilation of Java applications into native binaries (1).

Modern JVMs such as those using the GraalVM compiler employ tiered compilation (2), where methods progress through multiple compilation levels of increasing optimization aggressiveness. The JVM profiles methods during interpretation and uses the collected runtime information to guide the compiler in producing optimized native code.

Crucially, because the compiler and the application share the same execution resources (processor and memory), the timing of compilation directly affects the performance observed during benchmark execution. Early iterations of a benchmark workload execute partially interpreted or minimally optimized code, resulting in higher execution times. As compilation progresses, execution times decrease until the application reaches a state commonly referred to as *steady state*, where the most frequently executed methods have been compiled and optimized. However, even during steady state, the compiler may occasionally *de-optimize* previously compiled methods when runtime assumptions are invalidated, triggering recompilation and causing temporary performance disruptions.

Furthermore, as noted by Bulej et al. (1), profile-guided compilation is non-deterministic in principle. A single execution is not necessarily representative of the average performance observable with the same benchmark workload and compiler version across multiple executions.

1.2.2 Garbage Collection

The JVM manages memory automatically through garbage collection (GC). Periodically, the garbage collector identifies objects that are no longer reachable by the application and reclaims their memory. During a collection event, application threads may be paused or slowed down, causing a temporary increase in the measured iteration time.

GC activity is one of several sources of noise in benchmark measurements, alongside other runtime and environmental factors. Unlike compilation noise, which is concentrated in the warmup phase, GC pauses can occur at any point during execution, including during the steady-state phase. This makes GC-related noise particularly relevant for performance evaluation. The nature of this noise and its implications for normalization are analyzed in Chapter 2.

Different GC algorithms exhibit different pause characteristics. The dataset includes measurements under multiple GC configurations, allowing the analysis pipeline to identify whether a detected anomaly is GC-specific or is observed

across multiple configurations.

1.2.3 Typical Benchmark Execution Profile

A typical benchmark execution on the JVM can be divided into two phases (3). The *warmup phase* is characterized by high and variable iteration times as the JVM actively compiles hot methods. The *steady-state phase* begins once major compilation activity has subsided and iteration times stabilize, though they remain subject to occasional noise spikes. Traini et al. (3) note that some benchmarks may take a very long time to stabilize or may not stabilize at all. Figure 1.1 illustrates both phases across four benchmark runs from the dataset: the initial high iteration times during warmup followed by a more stable regime with periodic spikes.

For performance evaluation, only the steady-state iterations are relevant. The warmup phase reflects JVM runtime behavior rather than compiled code quality. Identifying the transition point is the task of the steady-state detector described in Chapter 2.

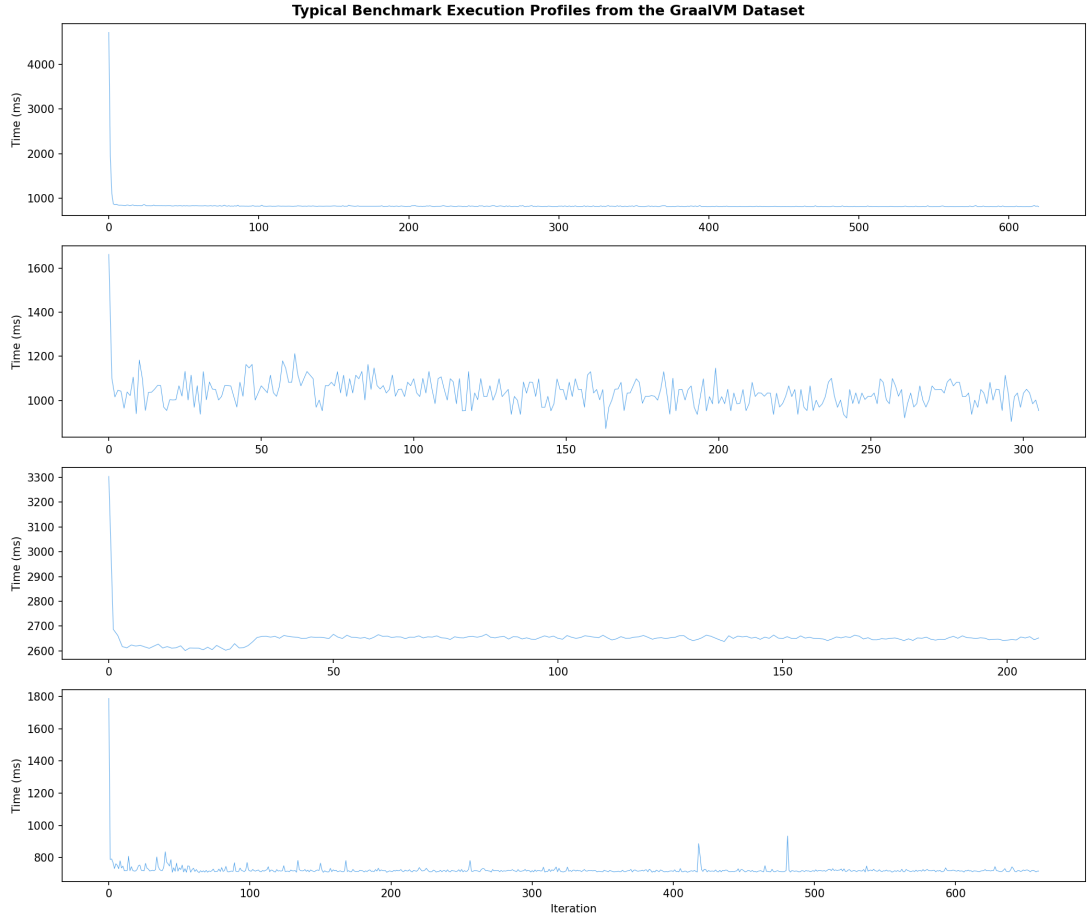


Figure 1.1 Raw iteration times for four benchmark runs from the GraalVM dataset. The warmup phase (high initial iteration times) and the subsequent steady-state phase with periodic noise spikes are visible across all four benchmarks.

1.3 The GraalVM Compiler Benchmark Results Dataset

The data used throughout this thesis comes from the GraalVM Compiler Benchmark Results Dataset, a publicly available data artifact presented by Bulej et al. (1). The dataset aggregates more than 70 machine time years of performance measurements collected over seven years of development of the GraalVM Compiler Project, from 2016 to 2022. It was created by researchers at the D3S department of Charles University together with engineers at Oracle Labs, with the explicit goal of supporting research into performance change detection methods and related problems. The benchmark workloads come from well-known suites including DaCapo (4), ScalaBench (5), a modified version of SPECjvm2008, and the Renaissance suite (6), chosen to provide representative compiler workloads. A detailed description of the measurement procedure, and experiment design can be found in the same work.

1.3.1 Dataset Structure

The measurement results are organized in a directory hierarchy under a `measurement/` directory, with each individual measurement occupying its own directory. To avoid storing too many entries at a single level, a three-level numerical directory structure is used.

Each measurement directory contains a `default.csv` file with one row per benchmark workload repetition (iteration). The features most relevant to this work are:

- `iteration_time_ns`: the wall-clock duration of the current iteration in nanoseconds, which serves as the primary performance metric throughout this thesis.
- `compilation_time_ms`: the aggregate execution duration of the compiler threads during the current iteration, used by the steady-state detector (Chapter 2) to identify when major compilation activity has subsided.

Additional features record hardware performance counters, memory usage, and garbage collection statistics, but these are not used in the current work.

1.3.2 Measurement Metadata

Each measurement directory also contains a `metadata` file, a JSON document with several fields. Each field is a numerical identifier that can be resolved against separate metadata files to obtain its meaning. The fields relevant to this work are:

- `benchmark_workload`: identifies the benchmark workload.
- `machine_host`: identifies the physical machine on which the measurement was executed.
- `platform_installation`: identifies the compiler installation, from which the platform type and compiler version can be derived.

- **configuration:** identifies the garbage collection configuration.

The full metadata schema and the relationships between these types can be found in the dataset paper. The specific resolution process used in this work is described in Section 1.4.2.

1.4 Configuration as the Unit of Analysis

The dataset contains measurements collected across many different benchmark workloads, physical machines, compiler build variants, and garbage collection settings. Not all of these measurements are directly comparable: a change in iteration time between two compiler versions could reflect an actual performance change, or it could simply be an artifact of running on a different machine or under a different GC configuration. To isolate the effect of compiler version changes, we need a structured way to group only those measurements that share the same experimental conditions. This is the role of the *configuration*.

1.4.1 Defining a Configuration

Throughout this thesis, a *configuration* is defined as a unique combination of four attributes:

1. **Benchmark type:** the name of the benchmark (e.g., `batik-small`).
2. **Machine host:** the identifier of the physical machine.
3. **Platform type:** the compiler build variant, combining the edition (Enterprise or Community), branch (master or release), and JDK version.
4. **GC configuration:** the garbage collection configuration.

Together, these four attributes represent the most granular level at which performance measurements can be meaningfully compared. Each attribute controls a different factor of the experimental setup: the workload being executed, the hardware it runs on, the compiler build variant producing the code, and the memory management strategy in use. We formalize the configuration as follows:

Definition. *Let m be a measurement. The configuration of m is the tuple $c(m) = (b, h, p, g)$ where b is the benchmark type, h is the machine host, p is the platform type, and g is the GC configuration.*

Property. *Two measurements m_1 and m_2 belong to the same configuration if and only if $c(m_1) = c(m_2)$.*

1.4.2 Metadata Resolution and Filtering

To construct a configuration from raw measurements, the metadata must first be resolved and filtered. Each measurement’s metadata file (Section 1.3.2) contains numerical identifiers that are resolved as follows:

1. From `benchmark_workload`, the workload *name* is extracted. The workload type is also available but is not used in this work.
2. From `platform_installation`, two values are derived: the *platform type* (identifying the build variant) and the *version* (the commit identifier of the compiler).
3. `machine_host` and `configuration` are used directly as numerical identifiers.

Since the dataset contains measurements for both JIT and AOT (Native Image) compilation modes, and this thesis focuses on JIT compilation (Section 1.2.1), the resolution process filters out all AOT-related platform types and GC configurations.

1.4.3 From Configuration to Time Series

As discussed in Section 1.2.1, profile-guided compilation is non-deterministic, and the dataset may therefore contain multiple measurements for the same configuration and compiler version. After applying deduplication (retaining only the measurement with the most recorded iterations), we obtain the following:

Corollary. *Two measurements belong to the same configuration if and only if they share all four attributes and differ only in the compiler version.*

Given measurements $m_1, m_2, \dots, m_n$ with corresponding compiler versions $v_1 < v_2 < \dots < v_n$ such that $c(m_i) = c(m_j)$ for all i, j , by the corollary above they belong to the same configuration $C = c(m_i)$ and differ only in their compiler version. We define the *configuration time series* as:

$$S_C = [(m_1, v_1), (m_2, v_2), \dots, (m_n, v_n)]$$

Each element (m_i, v_i) represents a benchmark measurement under configuration C at compiler version v_i . This sequence S_C is the fundamental input to the analysis pipeline described in this thesis.

1.5 Summary and Problem Formulation

This chapter introduced the background necessary for the remainder of this thesis. We described the sources of non-deterministic behavior in JVM benchmark execution, including JIT compilation (Section 1.2.1) and garbage collection (Section 1.2.2), and presented the GraalVM Compiler Benchmark Results Dataset (Section 1.3). We then defined the *configuration* as the unit of analysis and showed how, after metadata resolution, filtering, and deduplication, each configuration yields an ordered sequence of measurements:

$$S_C = [(m_1, v_1), (m_2, v_2), \dots, (m_n, v_n)]$$

Since the configuration is held constant across S_C , any performance change detected between consecutive elements (m_i, v_i) and (m_{i+1}, v_{i+1}) can be attributed to the version transition $v_i \rightarrow v_{i+1}$. The goal of this thesis is to automatically identify such transitions where a meaningful change in benchmark behavior has occurred. However, the raw iteration times in S_C are not yet suitable for analysis: they contain warmup artifacts and are subject to noise. Chapter 2 addresses how these raw measurements are prepared for the anomaly detection pipeline.

2 Preprocessing

As established in Chapter 1, the analysis pipeline operates on the configuration time series S_C , where each element (m_i, v_i) is a benchmark measurement at a specific compiler version. However, the raw iteration times recorded in each measurement are not directly suitable for analysis. They contain warmup artifacts from JIT compilation and are subject to noise from garbage collection, de-optimizations, and other runtime factors.

This chapter describes the two preprocessing steps applied to the raw data before it enters the anomaly detection pipeline: steady-state detection, which removes the warmup phase from each measurement, and normalization, which transforms the remaining iteration times into a representation that is robust to runtime variability and comparable across measurements.

2.1 Steady-State Detection

As described in Section 1.2.3, a benchmark execution on the JVM begins with a warmup phase during which the compiler is actively optimizing hot methods. The warmup iterations are not representative of the peak performance we are interested in analyzing, and including them would obscure the actual performance characteristics of the compiled code. The goal of steady-state detection (SSD) is to identify the iteration index at which warmup has ended and discard all preceding iterations.

The approach used in this work is based on monitoring the compilation activity signal, as suggested by Bulej et al. (1), who note that “waiting for a window of reduced compilation activity, as indicated by the reported compiler thread execution duration, provided reasonably reliable results.” Specifically, we use the `compilation_time_ms` feature, which records the aggregate execution duration of the compiler threads during each iteration. By detecting when this activity falls below a threshold, we identify the approximate start of the steady-state phase. As noted in Section 1.2.3, some benchmarks may never fully reach a stable steady state, so this detection is an approximation rather than a guarantee.

2.1.1 CAS Analysis (Compilation Activity Signal)

The Compilation Activity Signal (CAS) analysis operates on the compilation time series described above. The idea is straightforward: during warmup, the compiler is actively working and the compilation time per iteration is high. Once the most important methods have been compiled, this activity drops significantly compared to the warmup phase. The CAS analysis detects this transition.

The algorithm computes a rolling mean of the compilation time over a sliding window of size w and finds the first position where it falls at or below a threshold τ . This position is the cutoff index:

$$i^* = \min \left\{ k : \frac{1}{w} \sum_{j=k}^{k+w-1} x_j \leq \tau \right\}$$

Both w and τ are computed dynamically per measurement, as described in Section 2.1.2. All iterations before i^* are discarded. If no such index exists (compilation activity never subsides), the measurement is excluded from analysis.

Figure 2.1 illustrates the CAS analysis on a benchmark run from the dataset. The compilation time is high during the first iterations and then drops, with the rolling mean crossing the threshold at the detected cutoff point.

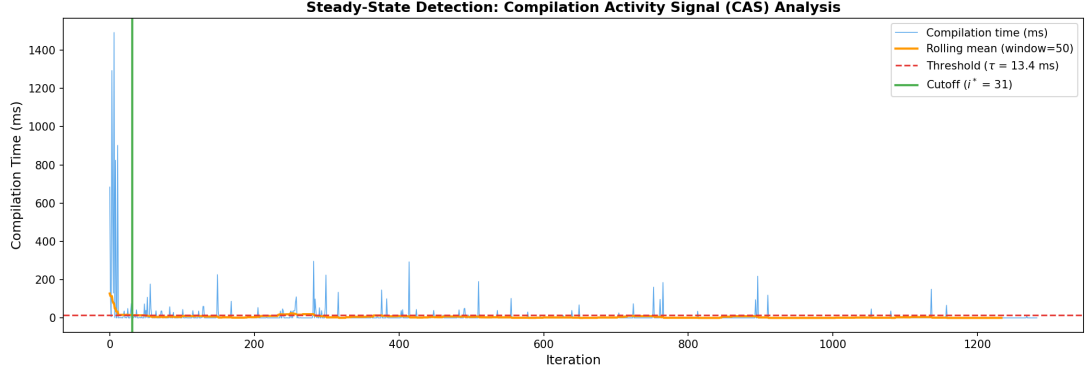


Figure 2.1 Steady-state detection on a benchmark run. The blue line shows the raw compilation time per iteration, the orange line shows the rolling mean, and the dashed red line indicates the threshold. The vertical green line marks the detected cutoff index i^* .

2.1.2 Dynamic Parameters

The CAS analysis requires two parameters: the window size w for the rolling mean, and the threshold τ against which the rolling mean is compared. Choosing appropriate values for these parameters is a challenging task, as the measurements in the dataset vary widely in length (from tens to hundreds of iterations) and in the magnitude and duration of their warmup phases. Fixed parameter values would inevitably be too aggressive for some benchmarks and too conservative for others.

To address this, both parameters are computed dynamically based on properties of each individual measurement. The specific formulas were determined through *expert judgement*, by manually evaluating different parameter choices across a variety of benchmarks with different execution profiles and selecting the values that yielded the most consistent results.

Window size w . The window size determines how many consecutive iterations are averaged when computing the rolling mean. It is set to 10% of the sequence length n , clamped to the range $[5, 50]$:

$$w = \text{clip}(0.10 \times n, 5, 50)$$

Averaging over a window rather than checking individual iterations prevents the algorithm from triggering on a single low-compilation iteration that happens to be surrounded by high ones.

Threshold τ . The threshold determines how low the rolling mean of compilation activity must drop before steady state is declared. Since different benchmarks exhibit different levels of compilation intensity, the threshold is defined relative to the maximum compilation time observed in the measurement:

$$\tau = 0.009 \times \max(x_1, x_2, \dots, x_n)$$

This means the algorithm waits until the rolling mean has dropped to 0.9% of the maximum, at which point compilation activity has effectively subsided. We observed that in the steady-state phase, compilation activity typically falls to less than 1–2% of its maximum. We therefore experimented with fractions in this range (0.5% to 1.5%) and found that 0.9% yielded quite some consistent cutoff points across different benchmarks. These values worked well for the measurements used in this thesis.

When applying this approach to other benchmark data, the parameters should be adjusted to suit the specific data at hand.

2.2 Normalization

2.2.1 Noise Characteristics in the Dataset

2.2.2 Why Standard Normalization Fails

2.2.3 Log-Transform and Robust IQR Scaling

2.3 Per-Configuration Preprocessing Pipeline

3 Related Work

3.1 Performance Regression Detection

3.1.1 Statistical Approaches

3.1.2 Machine Learning Approaches

3.2 Alternative Approaches

3.2.1 Statistical Feature Extraction

3.2.2 LSTM Autoencoders

3.3 TS2Vec and Contrastive Learning

3.4 Gap: Time Series Representations for Benchmark Analysis

4 TS2Vec and Contrastive Learning

5 Anomaly Detection Pipeline and Reporting

Conclusion

Bibliography

1. BULEJ, Lubomír; HORKÝ, Vojtěch; TUCCI, Michele; TŮMA, Petr; FARQUET, François; LEOPOLDSIEDER, David; PROKOPEC, Aleksandar. GraalVM Compiler Benchmark Results Dataset (Data Artifact). In: *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE '23 Companion)*. New York, NY, USA: ACM, 2023, pp. 65–69. Available from DOI: 10.1145/3578245.3585025.
2. HARTMANN, Tobias; NOLL, Albert; GROSS, Thomas. Efficient Code Management for Dynamic Multi-Tiered Compilation Systems. In: *Proceedings of PPPJ 2014*. 2014. Available from DOI: 10.1145/2647508.2647513.
3. TRAINI, Luca; CORTELLESA, Vittorio; DI POMPEO, Daniele; TUCCI, Michele. Towards Effective Assessment of Steady State Performance in Java Software: Are We There Yet? *Empirical Software Engineering*. 2022, vol. 28, no. 1, p. 13. Available from DOI: 10.1007/s10664-022-10247-x.
4. BLACKBURN, Stephen M.; GARNER, Robin; HOFFMANN, Chris; KHANG, Asjad M.; MCKINLEY, Kathryn S.; BENTZUR, Rotem; DIWAN, Amer; FEINBERG, Daniel; FRAMPTON, Daniel; GUYER, Samuel Z.; HIRZEL, Martin; HOSKING, Antony; JUMP, Maria; LEE, Han; MOSS, J. Eliot B.; PHANSALKAR, Aashish; STEFANOVIĆ, Darko; VANDRUNEN, Thomas; DINCKLAGE, Daniel von; WIEDERMANN, Ben. The DaCapo Benchmarks: Java Benchmarking Development and Analysis. In: *Proceedings of OOPSLA 2006*. 2006. Available from DOI: 10.1145/1167473.1167488.
5. SEWE, Andreas; MEZINI, Mira; SARIMBEKOV, Aibek; BINDER, Walter. Da Capo Con Scala: Design and Analysis of a Scala Benchmark Suite for the Java Virtual Machine. In: *Proceedings of OOPSLA 2011*. 2011. Available from DOI: 10.1145/2048066.2048118.
6. PROKOPEC, Aleksandar; ROSÀ, Andrea; LEOPOLDSIEDER, David; DUBOSCQ, Gilles; TŮMA, Petr; STUDENER, Martin; BULEJ, Lubomír; ZHENG, Yudi; VILLAZÓN, Alex; SIMON, Doug; WÜRTHINGER, Thomas; BINDER, Walter. Renaissance: Benchmarking Suite for Parallel Applications on the JVM. In: *Proceedings of PLDI 2019*. 2019. Available from DOI: 10.1145/3314221.3314637.

List of Figures

1.1	Raw iteration times for four benchmark runs from the GraalVM dataset. The warmup phase (high initial iteration times) and the subsequent steady-state phase with periodic noise spikes are visible across all four benchmarks.	10
2.1	Steady-state detection on a benchmark run. The blue line shows the raw compilation time per iteration, the orange line shows the rolling mean, and the dashed red line indicates the threshold. The vertical green line marks the detected cutoff index i^*	15

List of Tables

List of Abbreviations

A Attachments

A.1 First Attachment